Few-Label Transfer Across Network Flow Datasets: Meta-Training on Network Tasks Versus a Tabular Foundation Model

Quang-Vinh Dang
British University Vietnam, Hung Yen, Vietnam
vinh.dq4@buv.edu.vn ORCID 0000-0002-3877-8024

Abstract

Flow-based intrusion detectors that score above 0.99 ROC-AUC inside one dataset often fall to chance on another. On eight public flow datasets from five laboratories, we find that this is not a tuning problem: no flow feature keeps the same direction of association with the attack label across datasets, and a classifier trained on the pooled other datasets reaches only 0.32 to 0.95 ROC-AUC on a held-out one. In our pairwise benchmark, domain-adaptation methods and our own transfer method did not improve on a plain gradient-boosted baseline, which is consistent with a change in $P(y \mid x)$ between datasets. We therefore ask what can be learned from other networks when only $k = 4$ to 128 target flows are labelled. We build FLARE-2, a transformer meta-trained on episodes drawn from the other datasets, and compare it with tree ensembles and with TabPFN versions 3 and 3.5, general-purpose tabular foundation models that we did not train or tune on network data. It reads the detection rule from the labelled flows given at test time instead of storing a rule, so no model is trained or tuned on the target; its per-episode rank normalisation does use the unlabelled target flows. Each episode uses a random label space (binary, one class against benign, class against class, or N-way), random class-to-slot assignments, per-episode rank normalisation, and random feature sign flips; the model has no feature identity and a class-equivariant matching readout. Checkpoints are selected on a held-out source dataset. Over the eight targets, FLARE-2 shows no significant difference from random forest and CatBoost on binary ROC-AUC (mean over three meta-training seeds: -0.001 to target-only CatBoost, 95% interval $[-0.004, +0.002]$; $+0.001$ to random forest, $[-0.004, +0.006]$), and is better than logistic regression and kNN on every target. Both TabPFN versions are better than FLARE-2 on every target (mean difference to TabPFN-3.5 -0.014 over three seeds, 95% interval $[-0.018, -0.011]$) and better than the tree ensembles, so meta-training on real network tasks does not beat a strong general-purpose prior at binary detection. At attack-class level, averaged over three meta-training seeds, it has higher per-class ROC-AUC than random forest and CatBoost on 7 of 8 targets (mean $+0.011$, $p_{\text{Holm}} = 0.047$) and a lower false-positive rate on 7 of 8 targets ($p_{\text{Holm}} = 0.047$), at lower recall on all 8 targets at the default threshold ($p_{\text{Holm}} = 0.031$); N-way macro-F1 is not significantly different. We report where FLARE-2 does not help: it does not outperform tree ensembles fitted on the target labels at $k \geq 32$ or on easy targets, source-pretrained CatBoost is better at $k = 4$ on four targets, and TabPFN is better on all targets. Code, configurations and all raw per-run results are public.

1 Introduction

Machine-learning intrusion detection on network flows is usually evaluated by splitting one dataset into train and test. Under that protocol, gradient-boosted trees reach ROC-AUC above 0.99 on CIC-IDS2017, USB-IDS-1 and similar sets. Studies that move the same model to a second dataset report a collapse [2, 5, 8, 17], and a line of work asks whether domain adaptation helps [18]. An earlier study of ours trained on one dataset, measured the distance of a new dataset to it with an

isolation-forest score, and fine-tuned CatBoost on labelled target rows [7]. That approach needs target labels, and we show below that, on average, source pre-training does not beat training on the target labels alone.

This paper builds on the diagnosis and asks what source data can still contribute when a few target flows are labelled. We make four contributions.

1. **Evidence that cross-dataset failure is concept shift.** On eight datasets from five laboratories and three flow extractors, no common flow feature keeps its sign of association with the attack label (0 of 14), a domain classifier recognises the source dataset of a flow with 91.5% accuracy among the seven core datasets, and a pooled-source CatBoost reaches 0.32–0.95 ROC-AUC on the held-out dataset.

2. **FLARE-2**, a few-label meta-learner for flow data trained on tasks from other networks. It is permutation-invariant over features, accepts any feature set, and handles binary detection and attack-type classification with the same readout.

3. **A comparison with a tabular foundation model, which is the main empirical finding.** TabPFN versions 3 and 3.5, general-purpose models that we did not train or tune on network data (the model card of version 3 states that it is trained purely on synthetic tabular tasks [22]), are better than FLARE-2 and than tree ensembles on binary ROC-AUC on every target, by about 0.01 to 0.02.

4. **A paired evaluation at the level of the target dataset and the attack class**, with exact Wilcoxon tests, bootstrap intervals over targets, three meta-training seeds, controls for the normalisation and for in-context behaviour, and a candid account of the cases FLARE-2 loses.

We do not claim that FLARE-2 is the best few-label method: it matches fixed-setting tree ensembles on binary ROC-AUC and a general-purpose foundation model is better. What the study supports is that a few labelled flows of the target network, combined with a strong pretrained prior, are the reliable ingredient, and that pretraining on real network tasks adds nothing measurable to that prior at the binary level.

2 Related work

Cross-dataset intrusion detection. Layeghy and Portmann [17] train on one NetFlow-v2 dataset and test on another and report attack-class F1 below 1% for several pairs; Cantone et al. [5] report MCC around 0.3 between CIC-IDS2017 and CIC-IDS2018; D'hooge et al. [8, 9] study generalisation per attack class; Apruzzese et al. [2] cross-evaluate detectors across datasets and Engelen et al. [11] document labelling errors in CIC-IDS2017; Arp et al. [4] and Apruzzese et al. [3] discuss pitfalls of ML evaluation in security. Sarhan et al. [24] propose standard NetFlow feature sets to ease cross-dataset use. Domain-invariant training [18], built on adversarial alignment [13] or covariance alignment [27], assumes the labelling function is shared. Our concept-shift controls test that assumption directly.

Few-shot and in-context learning. Matching and prototypical networks [26, 28] and gradient-based meta-learning [12] learn from episodes. Prior-fitted networks [21] and TabPFN [16] fit a transformer to synthetic tabular tasks and classify by attention to the labelled rows. FLARE-2 follows this in-context recipe but meta-trains on *real* tasks from other networks, adds a task

Table 1: Datasets. Extractor: CIC = CICFlowMeter. "Classes" counts attack classes with enough test rows for class-level evaluation.

Dataset	Laboratory	Extractor	Reference
CIC-IDS2017	UNB CIC	CIC	[25]
CIC-IDS2018	UNB CIC / CSE	CIC	[25]
USB-IDS-1	Univ. of Sannio	CIC	[6]
InSDN	Univ. College Dublin	CIC	[10]
UNSW-NB15	UNSW Canberra	Argus	[20]
ToN-IoT	UNSW Canberra	Zeek	[1]
IoT-23	Stratosphere Lab, CTU	Zeek	[15]
CTU-13	Stratosphere Lab, CTU	Argus	[14]

distribution designed for the cross-network case, and uses a matching readout that fits a variable label space. We also evaluate TabPFN itself, versions 3 and 3.5, on the same supports, which tests whether meta-training on network tasks adds anything to a model pretrained on synthetic tasks only. Set-based architectures [19] motivate our permutation invariance over rows and features.

3 Problem, data and evidence of concept shift

3.1 Datasets and common flow schema

Table 1 lists the eight datasets. CIC-IDS2017 and CIC-IDS2018 share a laboratory and extractor, so we report the benchmark across five laboratories and three extractors (CICFlowMeter, Argus, Zeek). Extractors export different features, so we keep only features of the same physical meaning and units: five base features (duration, forward and backward packets, forward and backward bytes) plus nine derived rates and ratios, all passed through $\log(1 + x)$. CTU-13 supports only a nine-feature minimal tier (four base features and five derived); FLARE-2 takes it as a target without change because it accepts any number of features. Ports and protocol are excluded. Rows are sampled by a seeded 64-bit hash of the harmonised features and label (a bottom-k sketch), which gives a uniform sample of distinct rows and preserves the natural attack prior.

3.2 Setting

We use leave-one-dataset-out (LODO): the target dataset is never seen in meta-training or checkpoint selection. At test time a method receives a support set of $k \in \{4, 8, 16, 32, 64, 128\}$ labelled target flows with at least one flow per class, and may use no other target label. FLARE-2 additionally ranks features over the unlabelled target rows it scores (Sect. 4); the baselines receive no such information. Support sets are drawn 10 times per (target, k) with fixed seeds and are identical for every method. Evaluation is on the target test split. For class level we use $k \in \{2, 4, 8, 16\}$ labelled flows *per class* and 5 draws.

3.3 Why transfer fails: three controls

No stable feature direction. For each of the 14 common features we compute the ROC-AUC against the attack label inside every dataset. None has AUC above 0.5 in all datasets or below 0.5 in all datasets (0 of 14 consistent; e.g. `total_bytes` ranges from 0.025 to 0.760).

Table 2: Zero-shot ROC-AUC on the target test split ($k = 0$). LODO: CatBoost on all other datasets pooled. Pairwise: mean and best single source from the pairwise benchmark (not computed for CTU-13). Full label: CatBoost trained on the full target train split.

Target	CatBoost LODO	pairwise mean	pairwise best	full label
CIC-IDS2017	0.832	0.559	0.761	0.999
CIC-IDS2018	0.656	0.547	0.698	0.968
CTU-13	0.358	–	–	0.962
InSDN	0.730	0.598	0.827	0.998
IoT-23	0.322	0.370	0.460	0.950
ToN-IoT	0.608	0.593	0.942	0.999
UNSW-NB15	0.485	0.585	0.781	0.977
USB-IDS-1	0.952	0.695	0.965	1.000

Datasets are identifiable. A CatBoost domain classifier recognises the source dataset of a flow with 91.5% accuracy on seven datasets (chance 14.3%), 91.8% on benign flows alone. A model given unlabelled context can therefore recognise which dataset it is in and apply a memorised rule; our first attempt with unlabelled context rows failed for this reason (Sect. 4).

Source pre-training does not help the target. Over 42 source–target pairs, a source-pretrained CatBoost fine-tuned on 0.1% of target labels reaches 0.835 ROC-AUC, below the 0.928 of CatBoost trained on those labels alone; with 300 or 500 extra trees the gap narrows (0.911 vs 0.932) but does not close. Table 2 shows the zero-shot picture: a CatBoost trained on all other datasets (LODO) scores from 0.322 (IoT-23) to 0.952 (USB-IDS-1), below chance on three targets (IoT-23, CTU-13, UNSW-NB15).

4 Method

FLARE-2 classifies a query flow by attending to labelled support flows of the same episode. At test time the episode is the k support flows and the query flows; nothing is trained.

Episode normalisation. Within an episode, each feature is mapped to its Gaussian rank over support and query rows and clipped to $[-4, 4]$. At test time the query rows are the whole unlabelled target test split (and, for per-class evaluation, only the rows of the class and of benign), whereas training episodes rank over 256 query rows, so FLARE-2 is transductive in the target features and the per-class pool depends on the class labels of the test rows. A control with a reference of 256 query rows, as in training, gives the same ROC-AUC (Sect. 7.1). Absolute scales, which differ across extractors, therefore never reach the network. Missing features are absent tokens.

Tokens and layers. Flow i with features $x_{i1}, \ldots, x_{id}$ becomes d feature tokens $t_{ij} = \mathrm{MLP}_v(x_{ij})$ with a shared scalar embedding, plus one label token. There is no feature-identity embedding, so the model cannot memorise that "feature 7 high means attack" and is invariant to feature order. Six layers of width 128 with 4 heads alternate *feature attention* (tokens of one row attend to each other) and *row attention* (rows attend across the episode). Support rows attend to support rows; a query row attends to support rows and itself only, so attention does not depend on which other queries share the batch; the rank normalisation above does.

Label slots and matching readout. Class identities differ between episodes, so each support class is given a random one of $M = 16$ label-slot embeddings as input. The readout does not index slots. Each row is summarised by the layer-normalised mean r_i of its tokens; a query gives $q = W_q r$, a support row s gives key $k_s = W_k r_s$, and the logit of an active class j with support S_j is

$$\ell_j = \log \sum_{s \in S_j} \exp\Big(\frac{\langle q, k_s \rangle}{\sqrt{w}\,\tau}\Big) - b \log |S_j|, \tag{1}$$

with $w = 128$, a learned temperature $\tau > 0$ and a learned $b \in [0, 1]$ that interpolates between sum and mean pooling over a class's keys. A softmax over the N active classes is trained with cross-entropy. The head is equivariant to permutations of classes and slots by construction and serves binary detection ($N = 2$) and attack-type classification alike. A slot-indexed linear head plateaued in training and was replaced by this one. At test time we average class logits over 8 members with different sign flips and class-to-slot maps.

Meta-training episodes. Each episode draws a source dataset (weighted by $\sqrt{\text{size}}$) and a *task*: binary attack vs benign (probability 0.25), one attack class vs benign (0.25), class vs class (0.15), or N-way with $N \sim U\{3..6\}$ among benign and attack classes (0.35). Raw attack types with fewer than 30 rows are dropped. The per-class support size is log-uniform on $\{1, \ldots, 32\}$ with at most 160 support rows in total, so a binary episode never has more than 64 support rows while evaluation uses up to 128; the query class proportions are Dirichlet(1) with at least one query row per class, so no fixed prior can be learned; 30% of binary, one-class and N-way episodes (about 25% of all episodes) combine benign rows of one dataset with attack classes of another; with probability 0.5 a random subset of at least 5 features is kept; 2% of support labels are flipped; each feature is sign-flipped independently with probability 0.5. A monotone warp was also tried; it has no effect under rank normalisation and was dropped. Because both the meaning of a class and the sign of every feature change from episode to episode, the only reliable strategy is to read the rule from the support set. This is the design response to the concept shift of Sect. 3.3; Sect. 7 measures how much the individual choices, including the class-level tasks, contribute, and the evidence for them is modest.

Held-out-source model selection. Our first variant added unlabelled context rows from the target network. It reached 0.97 AUC on validation episodes from the meta-training datasets but fell to 0.66–0.76 on the unseen IoT-23 target: with datasets identifiable at 91%, six binary tasks are easy to memorise. In FLARE-2, one source dataset (rotating, never the target) is excluded from training and used only to select the checkpoint with the lowest validation loss. The unlabelled-context variant is not part of FLARE-2.

Training. AdamW (learning rate 3×10^{-4}, weight decay 0.01, cosine schedule, 1k warm-up steps), 20k steps, 8 episodes per step, 256 query rows, gradient clipping at 1.0, mixed-precision autocast (fp16 on a Colab T4 for the main runs, bf16 on a local RTX GPU for the ablation runs and extra seeds). One model is trained per target (LODO), about 1 hour each. The label-slot readout computes in float32 to avoid half-precision overflow.

5 Experimental protocol

Baselines on identical supports. Random forest (RF), CatBoost [23], logistic regression (LR) and kNN trained on the k target labels only; CatBoost pretrained on the sources and fine-tuned

on the k labels (the earlier approach); and, as zero-shot references, CatBoost trained on all other datasets and the pairwise single-source results. Baselines use fixed standard settings and no tuning (RF: 300 trees; CatBoost: 300 iterations, depth 6, learning rate 0.05; LR and 5-NN on standardised features; class-balanced weights), since selecting hyperparameters from a handful of labels is itself unreliable; a baseline tuned with extra target labels could be stronger. For N-way, baselines are the multi-class versions trained on the labelled k per class.

Tabular foundation models. TabPFN [16], versions 3 and 3.5 (released weights of the `tabpfn` package, version 9.2.0, and of the `Prior-Labs/tabpfn_3` repository), with default settings and the same support and query rows as every other method. We did not train or tune the models on network data (the model card of version 3 states that it is trained purely on synthetic tabular tasks [22]; we have not verified the pretraining data of version 3.5); they read the support set as training data and score each query row independently of the other query rows, so, unlike FLARE-2, they do not use the unlabelled target rows. Version 3.5 was run for all three meta-training seeds and version 3 for seed 0. The weights are released under a non-commercial research license, which applies here.

Metrics and statistics. ROC-AUC (binary and per class), recall and false-positive rate at threshold 0.5, and macro-F1 for N-way classification. Draws of one target are not independent, so the unit of analysis is the *target*: the mean difference of FLARE-2 minus baseline over draws, then an exact two-sided Wilcoxon signed-rank test over the 8 targets (smallest attainable $p = 0.0078$), Holm-adjusted over the five baselines (four at class level, where we report uncorrected p), and a 95% bootstrap interval over targets. Only the mean over k is a primary endpoint; the $k \leq 16$ and $k \geq 32$ splits are exploratory. For class-level results we also report units of (target, class), with a warning that classes of one target are correlated; the target-level test is the conservative one.

6 Results

6.1 Binary detection

Table 4 and Fig. 1 show ROC-AUC versus k. With $k = 4$ labelled flows, FLARE-2 already beats the zero-shot pooled CatBoost on five of eight targets (CTU-13, IoT-23, ToN-IoT, UNSW-NB15, USB-IDS-1; Tables 2, 4); on the other three (CIC-IDS2017, CIC-IDS2018, InSDN), where pooled sources transfer well, 8 to 16 labelled flows are needed to catch up. Table 5 gives the paired statistics. Against the strongest baselines the difference is small and not significant: mean over k, FLARE-2 minus target-only CatBoost is -0.001 and minus RF is $+0.004$; both intervals contain zero and the win/loss counts over targets are 4/4 and 5/3. Against LR and kNN the gain is large: FLARE-2 is higher on all 8 targets for the mean over k ($+0.075$ and $+0.110$, $p_{\text{Holm}} = 0.039$ for both). Against source-pretrained CatBoost fine-tuned on the labels, seed 0 shows FLARE-2 better for $k \geq 32$ (7 of 8 targets, $p_{\text{Holm}} = 0.047$) and not different below; the repeat over seeds below shows that this advantage is not robust.

Three meta-training seeds. We repeated the benchmark (meta-training, the split of the target and the support draws) with seeds 1 and 2 on all eight targets, for FLARE-2 and every baseline. Table 3 gives the paired differences with the target as unit, after averaging each target over the three seeds. The differences to RF and to target-only CatBoost stay within ± 0.002 for the mean over k, and the standard deviation over seeds of the mean difference is 0.001 to 0.005, so training variance is small next to the spread between targets. FLARE-2 stays above LR and kNN on all 8 targets in

Table 3: FLARE-2 minus baseline in binary ROC-AUC, mean over meta-training seeds 0–2, unit = target (8). W/L: targets where FLARE-2 is higher / lower. SD: standard deviation over seeds of the mean difference. p_{Holm}: exact Wilcoxon over targets, Holm-adjusted over the five baselines within each scope. Only the mean over k is a primary endpoint.

Scope	Baseline	Mean diff.	95% CI	W/L	SD over seeds	p_{Holm}
mean over k	RF	+0.001	[-0.004, +0.006]	4/4	0.003	1.000
	CatBoost (target only)	-0.001	[-0.004, +0.002]	4/4	0.001	1.000
	CatBoost (source + fine-tune)	+0.002	[-0.015, +0.020]	3/5	0.004	1.000
	LR	+0.073	[+0.044, +0.102]	8/0	0.003	0.039
	kNN	+0.113	[+0.090, +0.135]	8/0	0.004	0.039
k≤16	RF	+0.002	[-0.006, +0.012]	3/5	0.003	1.000
	CatBoost (target only)	-0.002	[-0.008, +0.004]	3/5	0.001	1.000
	CatBoost (source + fine-tune)	-0.004	[-0.034, +0.030]	3/5	0.003	1.000
	LR	+0.082	[+0.046, +0.118]	8/0	0.005	0.039
	kNN	+0.174	[+0.131, +0.215]	8/0	0.009	0.039
k≥32	RF	-0.001	[-0.004, +0.004]	2/6	0.003	1.000
	CatBoost (target only)	-0.000	[-0.003, +0.002]	3/5	0.003	1.000
	CatBoost (source + fine-tune)	+0.007	[+0.002, +0.012]	5/3	0.005	0.328
	LR	+0.064	[+0.035, +0.095]	8/0	0.005	0.039
	kNN	+0.051	[+0.037, +0.066]	8/0	0.005	0.039

Table 4: Binary ROC-AUC, mean over 10 support draws. F2: FLARE-2; RF: random forest; CB: CatBoost on target labels; CB-ft: source-pretrained CatBoost fine-tuned on the labels. Bold marks the best of the four.

	$k = 4$					$k = 16$					$k = 64$				
Target	F2	RF	CB	CB-ft	TP3.5	F2	RF	CB	CB-ft	TP3.5	F2	RF	CB	CB-ft	TP3.5
CIC-IDS2017	0.785	0.718	0.754	**0.841**	0.747	0.861	**0.888**	0.861	0.880	0.887	0.965	0.966	0.969	0.956	**0.979**
CIC-IDS2018	0.605	0.599	0.604	**0.691**	0.611	0.695	0.657	0.711	**0.733**	0.713	0.892	0.884	0.880	0.875	**0.904**
CTU-13	0.599	0.599	0.599	0.470	**0.633**	0.704	0.700	**0.718**	0.648	0.709	0.797	0.806	0.797	0.783	**0.807**
InSDN	0.644	0.671	0.690	**0.804**	0.659	0.901	0.872	0.875	0.890	**0.911**	0.960	0.956	0.954	0.949	**0.970**
IoT-23	0.785	0.825	**0.839**	0.763	0.819	0.877	0.876	0.852	0.857	**0.886**	0.905	0.894	0.900	0.891	**0.911**
ToN-IoT	0.720	0.716	0.723	0.679	**0.767**	0.899	0.861	0.885	0.893	**0.910**	0.961	0.968	0.968	0.960	**0.975**
UNSW-NB15	0.647	0.679	0.682	0.625	**0.685**	0.794	0.809	0.788	0.767	**0.821**	0.870	0.875	0.871	0.859	**0.882**
USB-IDS-1	0.978	0.958	0.972	**0.983**	0.969	0.957	0.993	**0.997**	0.994	0.994	0.997	0.999	0.999	0.998	**1.000**

every scope ($p_{\text{Holm}} = 0.039$). The seed-0 advantage over fine-tuned CatBoost at $k \geq 32$ shrinks to +0.007 (5 of 8 targets, $p_{\text{Holm}} = 0.33$), and the mean difference over k is +0.002 (CI $[-0.015, +0.020]$). We therefore claim neither superiority nor inferiority to the tree ensembles on binary ROC-AUC.

Where FLARE-2 helps and where it does not is target-dependent (Fig. 2). It is ahead of RF at $k = 4$ on CIC-IDS2017 (+0.067) and ahead at $k = 8$–16 on InSDN and ToN-IoT, but behind at $k = 4$ on InSDN, IoT-23 and UNSW-NB15 and behind at $k = 16$ on USB-IDS-1 (-0.036) and CIC-IDS2017 (-0.027). Targets whose attacks are easy (USB-IDS-1, where pooled zero-shot CatBoost already reaches 0.952) leave little to gain.

6.2 Comparison with a tabular foundation model

Table 4 includes TabPFN-3.5. On seed 0 the mean ROC-AUC over targets and k is 0.862 for TabPFN-3.5, 0.857 for TabPFN-3, 0.850 for FLARE-2 and for target-only CatBoost, 0.846 for RF. TabPFN-3.5 is higher than FLARE-2 on all 8 targets (-0.012 for FLARE-2 minus TabPFN-3.5, $p_{\text{Holm}} = 0.016$ over the two versions; TabPFN-3: -0.008, $p_{\text{Holm}} = 0.016$), and the 3-seed difference to TabPFN-3.5 is -0.014 (95% interval $[-0.018, -0.011]$, 0 of 8 targets in favour of FLARE-2, standard deviation over seeds 0.002; Table 6). The sign holds at small and large k ($k \leq 16$: -0.016;

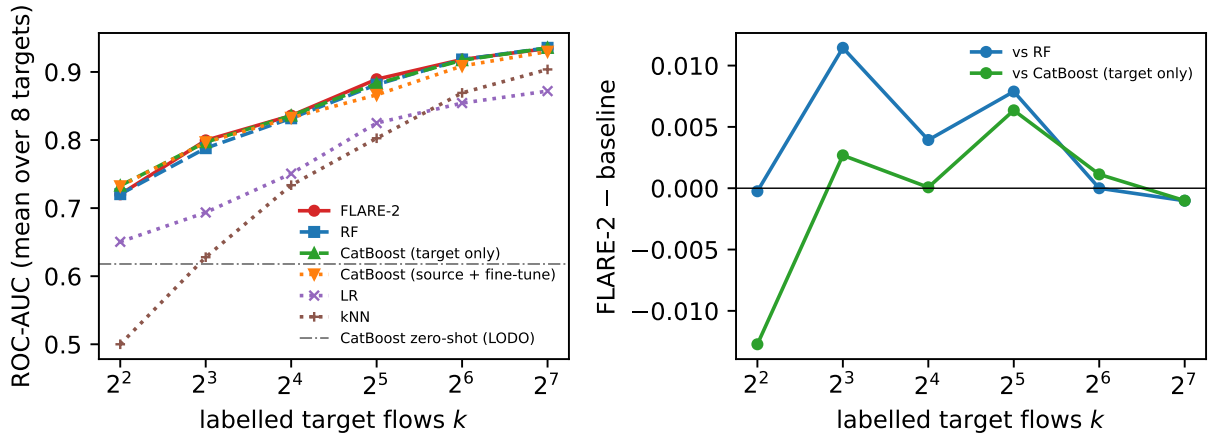


Figure 1: Left: ROC-AUC against k, mean over the eight targets. Right: FLARE-2 minus RF and minus target-only CatBoost.

$k \geq 32$: -0.013 over three seeds). TabPFN-3.5 is also above the tree ensembles on seed 0 ($+0.016$ over RF on 8 of 8 targets, $+0.012$ over target-only CatBoost on 7 of 8, uncorrected $p = 0.008$ and 0.016; these comparisons were added after the five-baseline family of Table 5 was fixed) and not different from source-pretrained CatBoost ($+0.017$, 5 of 8 targets, $p = 0.20$). The differences are small in absolute terms, at most about 0.02 ROC-AUC, but consistent in sign over targets, seeds and k. At $k = 4$ all methods but LR and kNN are within 0.02 of each other (0.720 for FLARE-2, 0.736 for TabPFN-3.5). Because TabPFN reads only the support rows while FLARE-2 additionally ranks features over the unlabelled target rows, the comparison does not favour TabPFN. We do not know why meta-training on five or six real networks is not better than pretraining on a broad synthetic prior with a larger model, and we have not tested explanations such as model scale, prior breadth or the number of meta-training datasets.

6.3 Attack-class level

Table 7 summarises per-class detection (class c against benign, k labelled flows of each) and N-way attack-type classification over 57 (target, class) detection tasks. FLARE-2 has higher per-class ROC-AUC than RF and CatBoost on 7 of 8 targets (mean $+0.014$ and $+0.015$ over targets, $p = 0.016$ before correction; the unit-level $p < 0.001$ is optimistic because classes within a target are correlated). It has a lower false-positive rate (by 0.05; lower on 7 of 8 targets, $p = 0.078$ at target level) and a lower recall (by 0.03 to 0.04; $p \leq 0.016$) at the default threshold of 0.5, so part of the ROC-AUC gain is a different operating point and not a free gain in detection. Repeating the class-level evaluation with meta-training seeds 1 and 2 and averaging each target over the three seeds (Holm-adjusted over the four baselines, unit = target) gives the same picture with firmer tests: per-class ROC-AUC is higher than RF and CatBoost by $+0.011$ on 7 of 8 targets ($p_{\text{Holm}} = 0.047$ for both), the false-positive rate is lower on 7 of 8 targets ($p_{\text{Holm}} = 0.047$) and recall is lower on all 8 targets ($p_{\text{Holm}} = 0.031$), so the operating-point caveat stands. The N-way macro-F1 differences to CatBoost and RF ($+0.018$ and $+0.021$, higher on 3 of 8 targets) are not significant. On N-way macro-F1 FLARE-2 is higher than RF and CatBoost on average ($+0.026$, $+0.022$) but on only 3 of 8 targets, so the difference is not significant; it is significantly higher than LR and kNN. Per-target values are in Table 8.

Table 5: Paired differences in binary ROC-AUC (FLARE-2 minus baseline), unit = target (8). W/L: targets where FLARE-2 is higher / lower. p_{Holm}: exact Wilcoxon over targets, Holm-adjusted over baselines.

Scope	Baseline	ΔROC-AUC	95% CI	W/L	p_{Holm}
mean over k	RF	+0.004	[-0.003, +0.010]	5/3	1.000
mean over k	CatBoost (target)	-0.001	[-0.004, +0.003]	4/4	1.000
mean over k	CatBoost (fine-tune)	+0.005	[-0.010, +0.022]	4/4	1.000
mean over k	LR	+0.075	[+0.044, +0.106]	8/0	0.039
mean over k	kNN	+0.110	[+0.085, +0.133]	8/0	0.039
k≤16	RF	+0.005	[-0.004, +0.014]	4/4	1.000
k≤16	CatBoost (target)	-0.003	[-0.009, +0.002]	4/4	1.000
k≤16	CatBoost (fine-tune)	-0.002	[-0.033, +0.030]	4/4	1.000
k≤16	LR	+0.087	[+0.047, +0.128]	8/0	0.039
k≤16	kNN	+0.165	[+0.121, +0.207]	8/0	0.039
k≥32	RF	+0.002	[-0.003, +0.008]	4/4	1.000
k≥32	CatBoost (target)	+0.002	[-0.002, +0.007]	4/4	1.000
k≥32	CatBoost (fine-tune)	+0.012	[+0.006, +0.019]	7/1	0.047
k≥32	LR	+0.064	[+0.034, +0.096]	8/0	0.039
k≥32	kNN	+0.056	[+0.040, +0.073]	8/0	0.039

Table 6: FLARE-2 minus TabPFN in binary ROC-AUC, unit = target (8). W/L: targets where FLARE-2 is higher / lower. Seed 0: p_{Holm} over the two TabPFN versions within each scope. Three seeds: TabPFN-3.5 only, target-averaged over seeds 0–2, exact Wilcoxon p without correction.

Seeds	Baseline	Mean diff.	95% CI	W/L	p_{Holm}
mean over k	seed 0, TabPFN-3	-0.008	[-0.011, -0.005]	0/8	0.016
	seed 0, TabPFN-3.5	-0.012	[-0.015, -0.009]	0/8	0.016
	3 seeds, TabPFN-3.5	-0.014	[-0.018, -0.011]	0/8	0.008
k≤16	seed 0, TabPFN-3	-0.008	[-0.014, -0.002]	1/7	0.055
	seed 0, TabPFN-3.5	-0.014	[-0.019, -0.009]	1/7	0.031
	3 seeds, TabPFN-3.5	-0.016	[-0.021, -0.012]	0/8	0.008
k≥32	seed 0, TabPFN-3	-0.008	[-0.010, -0.005]	0/8	0.016
	seed 0, TabPFN-3.5	-0.010	[-0.014, -0.007]	0/8	0.016
	3 seeds, TabPFN-3.5	-0.013	[-0.017, -0.009]	0/8	0.008

6.4 Ablations

7 Ablations

We retrain FLARE-2 with one component changed and evaluate on the same support draws as the main results (checked: the RF baseline reproduces the earlier numbers exactly). This costs one training run per cell, so we use a single meta-training seed and two targets (CTU-13 and CIC-IDS2017). The study therefore shows direction, not significance.

Table 9 shows four things. *Slot permutation.* Removing the random class-to-slot assignment matches FLARE-2 on CTU-13 (differences of at most 0.018 AUC over all k) but is unstable on CIC-IDS2017: binary AUC at k=16 falls to 0.368 and is below 0.5 in most draws at k=8 and k=16 (6 of 10 and 8 of 10 draws), while k=64 recovers to 0.942, still 0.023 below FLARE-2; its N-way macro-F1 at k=16 is not lower (0.567 against 0.504). A diagnostic on CIC-IDS2017 at k=16 (6 draws) explains the inversion: the model trained without permutation uses the index of

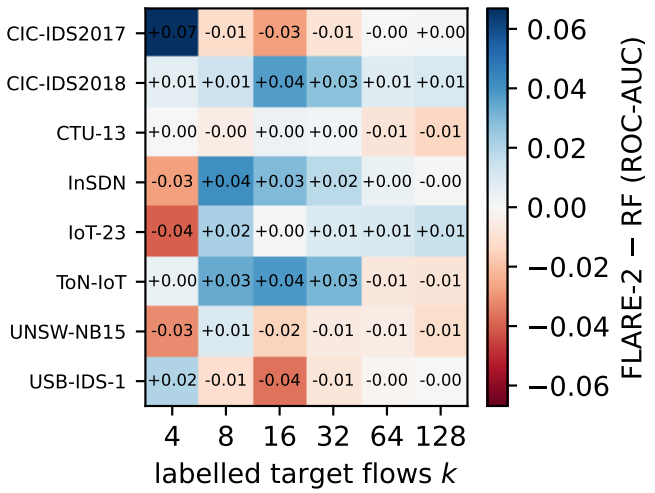


Figure 2: ROC-AUC difference FLARE-2 minus RF per target and k.

the label slot as a cue about the class. With the benign class in slot 0 it scores 0.443; with the two class ids swapped it scores 0.883, equal to the loss-masked model with random slots (0.882 in both orientations, as a class-equivariant model must). The ranking is therefore inverted by the orientation of the slots, not by noise, and random slot assignment removes this dependence. *Validation.* Selecting the checkpoint on in-source validation episodes instead of a held-out source changes binary AUC by at most 0.009 at $k \in \{4, 16, 64\}$ on both targets, by +0.048 at k=8 on CIC-IDS2017 (0.842 against 0.794) and raises N-way macro-F1 at k=16 on CIC-IDS2017 (0.562 against 0.504), so we cannot show that held-out validation matters on these two targets. *Loss mask.* Masking queries without support gives similar binary AUC on CTU-13 and higher AUC at $k \geq 16$ on CIC-IDS2017 (0.890 against 0.861 at k=16), with higher N-way macro-F1 on both targets. Trained on all eight targets (seed 0), the masked model has mean binary ROC-AUC 0.851 against 0.850 for FLARE-2, with per-target differences from −0.010 (InSDN) to +0.019 (CIC-IDS2018) and a gain on 4 of 8 targets, so the CIC-IDS2017 gain is not a general effect. *Class-level tasks.* Training on binary tasks only lowers binary AUC on CTU-13 by 0.01 to 0.04 and on CIC-IDS2017 by 0.01 to 0.05 at $k \geq 16$, and lowers N-way macro-F1 at k=16 on both targets (0.268 against 0.312 on CTU-13, 0.566 against 0.582 on CIC-IDS2017), while the per-class AUC at k=4 is nearly unchanged (0.858 against 0.863 and 0.959 against 0.963). Class-level tasks help binary AUC and N-way macro-F1 modestly, and we find no evidence that they drive per-class ranking.

7.1 Controls

These controls use the meta-training seed 1 models of all eight targets and the same support draws for every variant.

Does the transductive normalisation matter? The default ranks each feature over the support and all query rows of the target. Ranking over the support and a random subsample of 256 query rows, the size seen in meta-training, gives the same mean binary ROC-AUC (0.847 against 0.847; per-target differences between −0.003 and +0.003). Ranking over the support alone, with queries mapped through the support distribution, lowers it by 0.007 on average (lower on 6 of 8 targets,

Table 7: Class-level differences, FLARE-2 minus baseline. "targets": mean over the 8 targets of the target mean, count of targets where FLARE-2 is higher, exact Wilcoxon p over targets (uncorrected). "units": mean over (target, class) pairs, $k \in \{2, 4, 8, 16\}$ labelled flows per class. For FPR a negative difference favours FLARE-2.

Metric	Baseline	Δ (targets)	FLARE-2 higher	p	Δ (units)	units
Per-class ROC-AUC	RF	+0.014	7/8	0.016	+0.012	57
Per-class ROC-AUC	CatBoost	+0.015	7/8	0.016	+0.013	57
Per-class ROC-AUC	LR	+0.082	8/8	0.008	+0.075	57
Per-class ROC-AUC	kNN	+0.175	8/8	0.008	+0.167	57
Per-class recall@0.5	RF	-0.041	0/8	0.008	-0.044	57
Per-class recall@0.5	CatBoost	-0.026	1/8	0.016	-0.030	57
Per-class recall@0.5	LR	-0.025	2/8	0.078	-0.030	57
Per-class recall@0.5	kNN	-0.048	1/8	0.023	-0.056	57
Per-class FPR@0.5	RF	-0.053	1/8	0.078	-0.052	57
Per-class FPR@0.5	CatBoost	-0.052	1/8	0.078	-0.050	57
Per-class FPR@0.5	LR	-0.077	1/8	0.016	-0.074	57
Per-class FPR@0.5	kNN	-0.334	0/8	0.008	-0.330	57
N-way macro-F1	RF	+0.026	3/8	0.742	+0.026	8
N-way macro-F1	CatBoost	+0.022	3/8	0.742	+0.022	8
N-way macro-F1	LR	+0.074	7/8	0.016	+0.074	8
N-way macro-F1	kNN	+0.127	8/8	0.008	+0.127	8

Table 8: Per-target class-level results. Left: per-class ROC-AUC at $k = 4$ per class. Right: N-way macro-F1 at $k = 16$ per class.

	Per-class ROC-AUC, k=4			N-way macro-F1, k=16		
Target	F2	RF	CB	F2	RF	CB
CIC-IDS2017	**0.957**	0.942	0.949	**0.504**	0.456	0.448
CIC-IDS2018	**0.889**	0.879	0.875	**0.658**	0.517	0.582
CTU-13	**0.863**	0.853	0.852	**0.299**	0.329	0.350
IoT-23	**0.879**	0.842	0.841	**0.644**	0.562	0.589
ToN-IoT	**0.927**	0.906	0.925	**0.680**	0.705	0.688
UNSW-NB15	**0.857**	0.845	0.839	**0.294**	0.337	0.323
USB-IDS-1	**0.995**	0.998	0.999	**0.461**	0.461	0.468

by up to 0.017). The use of unlabelled target rows therefore has a small effect, and the 256-row reference removes the dependence on the full test split.

Does the model read the rule from the support labels? At k=16 and k=64 (5 draws), permuting the support labels among the support rows gives ROC-AUC between 0.49 and 0.57 on every target (mean 0.54 at k=16, 0.51 at k=64), against 0.82 and 0.91 with the true labels. Using the support rows of another draw of the same target gives 0.81 and 0.91, so the model reads the rule from labelled rows of the target and not from the particular rows. Swapping the two class ids leaves the AUC unchanged, as expected for a class-equivariant readout. We did not test support sets from a different target.

8 Limitations

- The main tables and figures use meta-training seed 0 with 10 support draws per target; seeds 1 and 2 were run for all targets and are summarised in Table 3 and in the text for the class level (the class-level tables themselves are seed 0), and the ablations use one seed and two targets.

Table 9: Ablations, seed 0, 10 support draws per k for binary metrics; class-level metrics use 10 draws for the no-permutation, in-source and CIC-IDS2017 loss-mask rows and 5 for the others. Binary ROC-AUC at $k \in \{4, 16, 64\}$, N-way macro-F1 at k=16 per class, and per-class ROC-AUC at k=4. Rows are compared with their own reference: the first three rows with FLARE-2, the last two with the loss-masked model.

	CTU-13					CIC-IDS2017				
Variant	k=4	k=16	k=64	F1@16	AUC@4	k=4	k=16	k=64	F1@16	AUC@4
FLARE-2 (reference)	0.599	0.704	0.797	0.299	0.863	0.785	0.861	0.965	0.504	0.957
no slot permutation	0.615	0.714	0.788	0.300	0.867	0.709	0.368	0.942	0.567	0.957
in-source validation	0.604	0.703	0.794	0.310	0.862	0.784	0.870	0.965	0.562	0.959
loss mask (reference)	0.594	0.707	0.792	0.312	0.863	0.775	0.890	0.969	0.582	0.963
loss mask, binary tasks only	0.563	0.671	0.779	0.268	0.858	0.776	0.856	0.949	0.566	0.959

- FLARE-2 is not better than fixed-setting tree ensembles on binary ROC-AUC, and TabPFN is better than FLARE-2 on every target; its remaining gains are class-level and conditional on the operating point.
- The TabPFN comparison uses default settings, three seeds for version 3.5 and one for version 3, and the released weights under a non-commercial license; a tuned or fine-tuned TabPFN, or a tuned FLARE-2, could change the gaps, which are about 0.01 to 0.02 ROC-AUC.
- Eight datasets from five laboratories (UNB, Sannio, UCD, UNSW, CTU), so datasets are not independent units; results on other networks may differ.
- FLARE-2 normalises features by rank over the unlabelled target rows it scores, which is transductive in the target features (a 256-row query reference gives the same ROC-AUC and support-only ranks lower it by 0.007, Sect. 7.1), and the baselines get no such information; one meta-trained model per target is needed (leave-one-dataset-out), and each model sees only five or six training datasets. All eight targets were used while choosing the architecture and the task mix, so there is no untouched test set. Source-pretrained CatBoost is better than FLARE-2 at $k = 4$ on four targets.
- Queries whose class has no support row (label noise at $k = 1$) receive a target logit of -10^9. This produced training losses of order 10^6 and, in one binary-only ablation run, made validation loss pick a collapsed checkpoint (AUC about 0.50); checkpoint selection now scores supported queries only. The checkpoint selection of the main FLARE-2 runs is unaffected (their validation episodes contained no unsupported queries), but their training loss includes these queries (logged loss of order 10^6 to 10^7). The loss-masked variant, trained on all eight targets with seed 0, has mean binary ROC-AUC 0.851 against 0.850 for FLARE-2, with per-target differences between -0.010 and $+0.019$ and a gain on 4 of 8 targets (Sect. 7).
- Flows only; payload features and temporal context are not used.
- The datasets are public laboratory captures with their own labelling errors [11].

9 Conclusion

Concept shift, rather than tuning, appears to limit cross-dataset flow classification, so a model that carries a fixed decision function across networks is a poor target. What source data can still provide is a prior over how to learn a rule from a few labelled flows. FLARE-2, a transformer meta-trained

on real network tasks, matches fixed-setting tree ensembles on binary detection without fitting the network to the target and modestly improves per-class ranking, but a general tabular foundation model, TabPFN versions 3 and 3.5, is better on every target by about 0.01 to 0.02 ROC-AUC without any training or tuning on network data by us. The practical conclusion is that labelled flows from the target network, read by a strong pretrained prior, are what works, and that pretraining on a handful of real networks is not the source of that prior's value. Open problems are the gap to full-label training (0.95–1.00 ROC-AUC) at small k, tuning and fine-tuning the foundation model, and testing whether a meta-trainer with more networks or a larger model closes the gap.

Reproducibility. The package, configurations, sampled-row hashes and every raw per-run CSV are in `https://github.com/vinhqdang/cross_ids`; tables and figures are generated by `scripts/make_paper_tables.py` and statistics by `scripts/stats_flare.py`.

References

[1] Abdullah Alsaedi, Nour Moustafa, Zahir Tari, Abdun Mahmood, and Adnan Anwar. TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems. *IEEE Access*, 8:165130–165150, 2020.

[2] Giovanni Apruzzese, Luca Pajola, and Mauro Conti. The cross-evaluation of machine learning-based network intrusion detection systems. *IEEE Transactions on Network and Service Management*, 19(4):5152–5169, 2022. doi: 10.1109/TNSM.2022.3157344. arXiv:2203.04686.

[3] Giovanni Apruzzese, Pavel Laskov, and Johannes Schneider. SoK: Pragmatic assessment of machine learning for network intrusion detection. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, pages 592–614, 2023. doi: 10.1109/EuroSP57164.2023.00042.

[4] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don'ts of machine learning in computer security. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3971–3988, 2022. arXiv:2010.09470.

[5] Marco Cantone, Claudio Marrocco, and Alessandro Bria. Machine learning in network intrusion detection: A cross-dataset generalization study. *IEEE Access*, 12:144489–144508, 2024. doi: 10.1109/ACCESS.2024.3472907. arXiv:2402.10974, titled "On the Cross-Dataset Generalization of Machine Learning for Network Intrusion Detection".

[6] Marta Catillo, Andrea Del Vecchio, Luciano Ocone, Antonio Pecchia, and Umberto Villano. USB-IDS-1: a public multilayer dataset of labeled network flows for IDS evaluation. In *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, pages 1–6, 2021. doi: 10.1109/DSN-W52860.2021.00012.

[7] Quang-Vinh Dang. Learning to transfer knowledge between datasets to enhance intrusion detection systems. In *Computational Intelligence*, Lecture Notes in Electrical Engineering, pages 39–46. Springer Nature Singapore, 2023. doi: 10.1007/978-981-19-7346-8_4.

[8] Laurens D'hooge, Tim Wauters, Bruno Volckaert, and Filip De Turck. Inter-dataset generalization strength of supervised machine learning methods for intrusion detection. *Journal of Information Security and Applications*, 54:102564, 2020. doi: 10.1016/j.jisa.2020.102564.

[9] Laurens D'hooge, Miel Verkerken, Tim Wauters, Filip De Turck, and Bruno Volckaert. Characterizing the impact of data-damaged models on generalization strength in intrusion detection. *Journal of Cybersecurity and Privacy*, 3(2):118–144, 2023. doi: 10.3390/jcp3020008.

[10] Mahmoud Said Elsayed, Nhien-An Le-Khac, and Anca Delia Jurcut. InSDN: A novel SDN intrusion dataset. *IEEE Access*, 8:165263–165284, 2020.

[11] Gints Engelen, Vera Rimmer, and Wouter Joosen. Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. In *2021 IEEE Security and Privacy Workshops (SPW)*, pages 7–12, 2021. doi: 10.1109/SPW53761.2021.00009.

[12] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.

[13] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59):1–35, 2016.

[14] Sebastián García, Martin Grill, Jan Stiborek, and Alejandro Zunino. An empirical comparison of botnet detection methods. *Computers & Security*, 45:100–123, 2014.

[15] Sebastian Garcia, Agustin Parmisano, and Maria Jose Erquiaga. IoT-23: A labeled dataset with malicious and benign IoT network traffic, 2020. Stratosphere Laboratory, Czech Technical University in Prague. Zenodo, doi:10.5281/zenodo.4743746.

[16] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025. doi: 10.1038/s41586-024-08328-6.

[17] Siamak Layeghy and Marius Portmann. Explainable cross-domain evaluation of ML-based network intrusion detection systems. *Computers and Electrical Engineering*, 108:108692, 2023. doi: 10.1016/j.compeleceng.2023.108692. arXiv:2205.04112, titled "On Generalisability of Machine Learning-based Network Intrusion Detection Systems".

[18] Siamak Layeghy, Mahsa Baktashmotlagh, and Marius Portmann. DI-NIDS: Domain invariant network intrusion detection system. *Knowledge-Based Systems*, 273:110626, 2023. doi: 10.1016/j.knosys.2023.110626. arXiv:2210.08252.

[19] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.

[20] Nour Moustafa and Jill Slay. UNSW-NB15: A comprehensive data set for network intrusion detection systems. In *Military Communications and Information Systems Conference (MilCIS)*, 2015.

[21] Samuel Müller, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka, and Frank Hutter. Transformers can do Bayesian inference. In *International Conference on Learning Representations*, 2022.

[22] Prior Labs. TabPFN-3: model card and license, 2026. URL `https://huggingface.co/Prior-Labs/tabpfn_3`. Hugging Face repository `Prior-Labs/tabpfn_3`, license of March 24, 2026. Accessed October 22, 2026.

[23] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems 31*, 2018.

[24] Mohanad Sarhan, Siamak Layeghy, and Marius Portmann. Towards a standard feature set for network intrusion detection system datasets. *Mobile Networks and Applications*, 27(1):357–370, 2022. doi: 10.1007/s11036-021-01843-0.

[25] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.

[26] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems 30*, 2017.

[27] Baochen Sun, Jiashi Feng, and Kate Saenko. Return of frustratingly easy domain adaptation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.

[28] Oriol Vinyals, Charles Blundell, Timothy Lillicrap, Koray Kavukcuoglu, and Daan Wierstra. Matching networks for one shot learning. In *Advances in Neural Information Processing Systems 29*, 2016.